



变量与数据类型

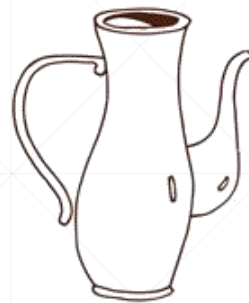
北京理工大学计算机学院
金旭亮

理解“变量”

程序运行所需要处理的数据，通常都需要放到“**变量 (variable)**”中。

变量可以看作是一种数据容器。

不同类型的“容器”，适合放置不同类型的
数据，这种“类型”，我们称它为“**变量的
数据类型**”。



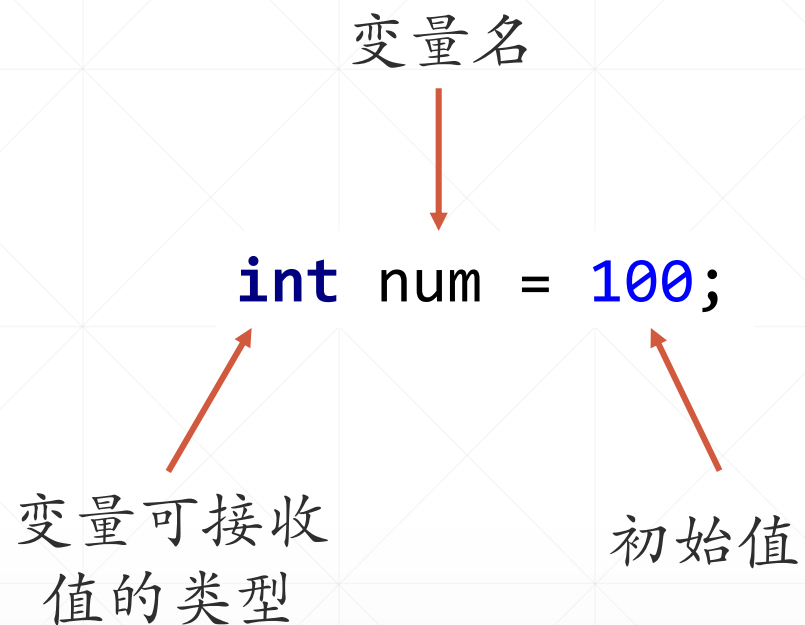
变量的定义格式

变量名

↓

int num = 100;

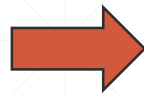
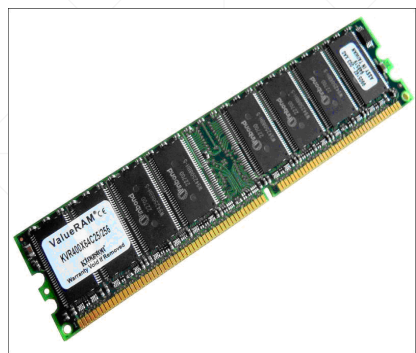
变量可接收值的类型 初始值



变量是一个数据的容器，在运行过程中，可以将不同的数值传给它。

变量“生活”在“内存 (memory)” 中.....

聊聊内存、内存单元和内存地址



内存单元 内存地址

	0000 0000
	0000 0001
	0000 0002
	.
	.
	.
	FFFF FFFF

应用程序中能使用的内存，是由操作系统提供的“**虚拟内存**”，其物理载体通常就是内存条和硬盘。

- 内存由多个内存单元构成，每个内存单元都有一个编号，称为“**内存地址**”。
- 给定一个“内存地址”，就能找到特定的内存单元。
- 计算机可以从内存单元中写入或读出数据。

使用比较底层的编程语言，比如“**汇编语言 (Assembly Language)**”，我们可以直接指定地址去存取特定的内存单元，但这么干“太低效了”，为什么？

不同的计算机硬件，可以访问的物理内存数量是变化的

不同的操作系统，甚至是同一操作系统的不同版本，对内存单元的存取可能都有着不同的要求



你必须针对特定的计算机去写“特定”的程序，硬件略有变化，如果不作修改，你的程序可能就崩溃了.....

计算机科学家给出的解决方案是.....

1

应用程序需要存取数据时，不指定具体的内存地址值，而只是给出一个“名字”，这个“名字”引用某块内存区域。

2

这个“名字”具体到底引用的是哪块内存区域，由操作系统（Windows）负责安排，应用程序就不要费这个心了！

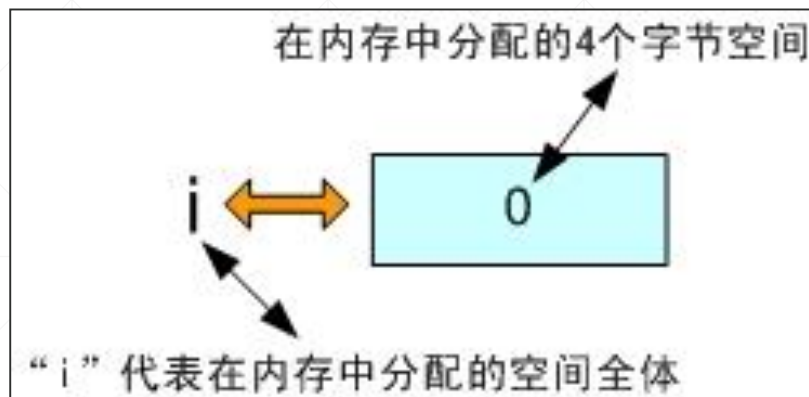
3

应用程序根本就不管它要用的数据具体保存在哪个地方，它只知道，我可以“按名”存取数据就够了！

这个内存区域的“名字”，就是我们所讲的“变量”！

通过变量名“间接”地存取内存

```
int i = 0 ;
```



- * 变量的名字（上面代码中的“i”），可以看成是特定内存区域的“别名”，通过它计算机就能找到特定的内存单元存取数据。
- * 有了变量名，就不再需要显式指定一长串的地址数值来访问内存单元了。

变量名

对应于

某块内存区域的“别名”

“给变量赋值”是什么意思？

变量定义好之后，可以随时地通过**赋值语句**传给它不同的值.....

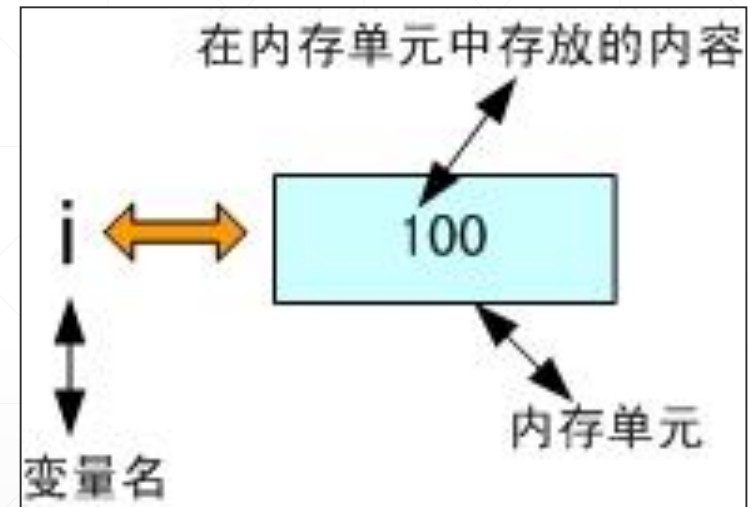
语法格式：

变量名 = 变量新值;

实例：

i = 100;

Java代码中的单个等号，称为“**赋值符**”，与数学中的等号是两回事，它表示要把等号右边的值，传给左边的变量。

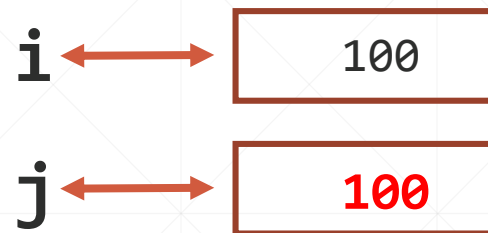


给变量**赋值**，其实就是找到变量所代表的内存区域，把指定的数值写入其中。

注意：写入到内存单元中的数据，都被转换为**二进制**数值，计算机并不能直接处理我们常用的十进制数。

变量之间的相互赋值，又该怎么理解？

```
int i = 100 ;  
int j;  
j = i;
```



- * 变量间的相互赋值，本质上是内存单元间的**值复制**。
- * 在上述代码中， $j = i$ ，其实就是把“i”中所保存的数值复制一份，然后保存到“j”所代表的内存单元中。
- * 注意：变量“i”和“j”之间是**完全独立的**，修改“i”的值，不会导致“j”的值同步变化。

在Java程序中使用常量与变量

```
public class ConstantAndVariable {  
  
    //常量定义  
    public static final int MAX_NUMBER = 255;  
  
    public static void main(String[] args) {  
        //变量定义  
        int num = 100;  
        //将变量num的值取出来，加1，再保存回变量num中  
        num = num + 1;  
        //变量和常量之间可以进行运算  
        System.out.println(num + MAX_NUMBER); //356  
    }  
}
```

表达式

参看ConstantAndVariable.java

常量与变量之间，可以通过运算符连接起来，构成“**表达式 (Expression)**”。

常量与变量的名字，称为“**标识符 (identifier)**”。

JDK10引入之新特性—隐式定义的局部变量

```
//使用var定义局部变量，是JDK10引入的特性
//变量的类型，由Java编译器依据其接收的值自动判断
//在本例中，intValue的类型为int。
var intValue = 100;
//以下语句将intValue变量加一，相当于
//intValue=intValue+1
intValue++;
int newValue = 101;
//输出: true
System.out.println(intValue == newValue);
```

```
//var所定义的变量，其类型不受限制
var info = "当前时间为: ";
var now = new Date();
//var所定义的变量，其用法与普通变量没什么两样
//当前时间为: Wed Sep 23 09:02:24 CST 2020
System.out.println(info + now.toString());
//以下两句调用JDK所提供之“反射”功能，提取特定变量的类型信息
//java.lang.String
System.out.println(info.getClass().getName());
//java.util.Date
System.out.println(now.getClass().getName());
```

参看示例：UseVar.java

重温一下Java对标识符的要求

1. 开头：字母，_ 和 \$
2. 不能以数字打头
3. 不能包含空白字符：制表, 空格, 回车, 换行符
4. 不能用关键字



“起名”很重要！

- 规范标识符的重要性:

一个职业软件开发者，应该保证写出规范的代码。

- 编程过程中时刻注意遵循Java标识符规范，

- 标识符起名应该尽量做到“望名知义”。对于中文语境下一些特殊的事物，如果很难译为英文，可以使用汉语拼音作为名字，但请使用全拼，并且，不要采用“拼音与英文单词混用”这种方式。



汇总：Java标识符的命名规范-1

除了前面PPT介绍过的，还有以下需要遵循的标识符命名规范：



包名：小写，需要保证唯一性，通常采用反写“互联网域名”的形式，比如，`cn.edu.bit.cs`



类名：应该选用名词，采用驼峰形式拼写，由多个单词拼接而成时，每个单词首字母大写。



接口名：命名规范与类名一致。其名字最好能体现出实现这一接口的类的特性，比如`Comparable`、`Serializeable`。

汇总：Java标识符的命名规范-2



方法名：小写字母打头，通常是“动宾结构”，从第二个单词起首字母大写，比如，addPlayer、getName。



泛型参数：通常采用大写的单字母，并且有以下含义：

E-Element（通常用于声明泛型集合）

K-Key

V-Value

S，U，V等等，通常用于定义除第一个以外的第二第三个泛型参数。

关键字 (Key words)

有一些标识符具有特定的含义，它们被称为“关键字”：

`int, for, class, extends, .....`



给变量、函数或类等起名字时，不能使用关键字。

原始数据类型与引用数据类型



Java中除了 `int` , `float` 等少数几个数据类型，其余的数据类型都用来**引用**对象，称为“**引用类型 (Reference Type)**”。



`int`、`float` 等这些数据类型称为“**原始数据类型 (Primitive type)**”。



这两种类型定义的变量，具有不一样的内存模型，非常重要，在学习时要注意区分，后面课程会详细介绍引用类型。

Java中一共有以下8种原始数据类型

boolean

byte

char

short

int

long

float

double



注意一下这八种数据类型，都是全小写字母。在Java中，如果某个数据类型是大写字母开头的，那它一定是引用类型。

`String`类型虽然非常常用，但注意它是引用类型，另外还请注意它名字的首字母是大写的，符合Java的命名规范。



介绍完了常量和变量的基础知识之后，下一讲，介绍Java中的运算符，以及由运算符、常量和变量所构成的表达式。